ESPRIT — École Supérieure Privée d'Ingénierie et de Technologie

Smart Freelance and Project Matching Platform

Ateliers 2, 3 et 4 — exigences, vue building block, vues de contexte, de déploiement et de runtime
12 octobre 2026

Les sept blocs portent le même nom dans tout le rapport. Les ports, bases et méthodes cités sont ceux du dépôt.

Bloc	Contenu retenu
Plateforme	Angular, API Gateway :8078, Eureka :8420, Config Server :8888
Identité et comptes	keycloak-auth :8079, user :8090 (userdb)
Mise en relation	project :8084, offer :8082, contract :8083, freelancia-job :8097
Profil et réputation	portfolio :8086, review :8085, gamification :8088
Exécution de mission	task :8091, planning :8081, chat :8096, meeting :8101
Support	notification :8098, ticket :8094, subcontracting :8110, vendor :8093
Aide à la décision	aimodel :8095, service ML :8102

1. Atelier 2 — Exigences fonctionnelles

Hiérarchie demandée : module, sous-module, fonctionnalité. La figure ne montre que les deux premiers niveaux. Les fonctionnalités sont les puces.

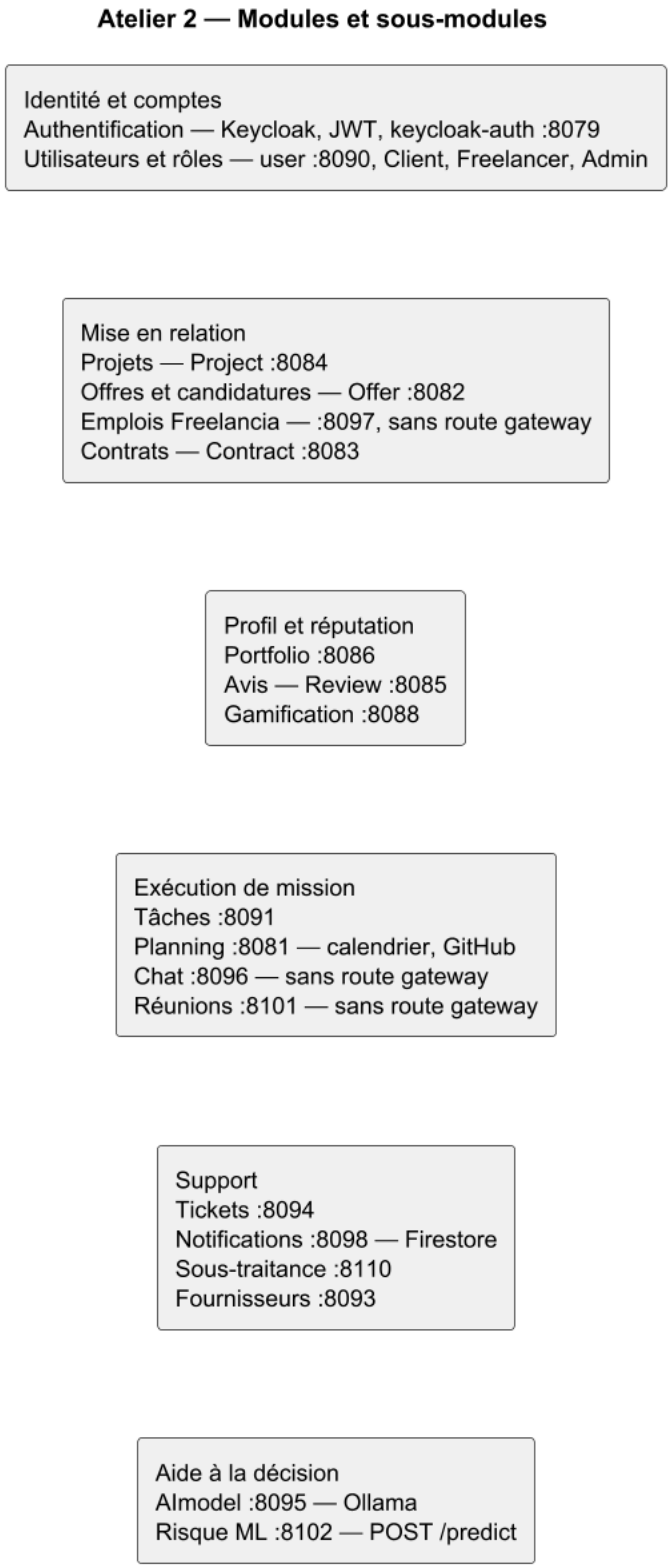


Figure 1. Modules et sous-modules.

Identité et comptes

Authentification.

- Inscrire un utilisateur dans le realm Keycloak *smart-freelance*.
- Connecter un utilisateur et obtenir un jeton JWT, via keycloak-auth (:8079), préfixe */keycloak-auth*.

Utilisateurs et rôles.

- Gérer le profil dans le service user (:8090, base userdb).

- Distinguer les rôles Client, Freelancer et Admin.

Mise en relation

Projets (Project :8084, base projectdb).

- Créer, modifier, supprimer et lister un projet (*addProject*, *updateProject*, *deleteProject*, *getAllProjects*).
- Consulter le risque, la satisfaction et le segment client (*getProjectRisk*, *getProjectSatisfaction*, *getClientSegment*).

Offres et candidatures (Offer :8082, base gestion_offre_db).

- Créer, publier, rechercher et recommander une offre.
- Postuler à une offre (*applyToOffer*). Cette étape n'envoie pas de notification.
- Accepter une candidature et créer un contrat brouillon (*acceptApplication*).
- Poser une question sur une offre et notifier le freelancer (*addQuestion*).
- Traduire le titre et la description via Google Translate.

Emplois Freelancia. Publier et consulter une annonce (FreelanciaJob :8097, base freelancia_job_db). Ce service n'a pas de route sur la passerelle.

Contrats. Enregistrer le contrat issu d'une candidature acceptée (Contract :8083, base gestion_contract_db).

Profil et réputation

- Portfolio (:8086) : enregistrer expériences et compétences.
- Avis (:8085) : noter, répondre, et notifier l'autre partie lorsqu'une réponse est créée.
- Gamification (:8088) : suivre niveaux, XP et succès.

Exécution de mission

- Tâches (:8091) : suivre les tâches et demander une génération au service AImodel.
- Planning (:8081) : publier un avancement, créer un événement Google Calendar, lire branches et commits GitHub.
- Chat (:8096) et réunions (:8101) : échanger et planifier. Aucune route gateway n'est configurée pour ces deux services.

Support

- Tickets (:8094) : ouvrir, lire et rouvrir un ticket.
- Notifications (:8098) : créer une notification (*POST /api/notifications*), stockée dans Firestore.
- Sous-traitance (:8110) : suivre une mission déléguée.
- Fournisseurs (:8093) : tenir le référentiel Vendor.

Aide à la décision

- AImodel (:8095) : générer un texte ou des tâches, puis appeler Ollama (:11434).
- Service ML (:8102) : estimer le risque d'un projet par *POST /predict*. Seul le service Project l'appelle.

2. Atelier 2 — Exigences non fonctionnelles

Une ligne marquée **cible** est un objectif de ce rapport. Une ligne marquée **constat** est une valeur déjà présente dans la configuration.

Exigence	Quantification	Statut
Temps de réponse des API métier, hors intelligence artificielle	≤ 2 s au 95e percentile	Cible
Timeout global du client HTTP de la passerelle	4 h (routes comprises, à cause du LLM local)	Constat
Timeout de la route planning	30 s	Constat
Timeout de la route portfolio	120 s	Constat
Timeout des routes IA (tâches AI et AImodel)	14 400 000 ms, soit 4 h	Constat
Réplicas de l'API Gateway et du frontend	2 réplicas chacun, maxUnavailable = 0	Constat
Taille de corps à l'ingress	50 Mo	Constat
Mémoire du codec de la passerelle	10 Mo	Constat
Lecture proxy de l'ingress	3 600 s	Constat
Authentification	OAuth2 / JWT, realm smart-freelance, rôles Client, Freelancer, Admin	Constat
Origines CORS du profil local	http://localhost:4200 et http://127.0.0.1:4200	Constat
Données relationnelles	Une base MySQL par microservice. Notification utilise Firestore, pas MySQL.	Constat
Accès direct aux microservices	Refusé sans l'en-tête X-Internal-Gateway posé par la passerelle	Constat

3. Atelier 3 — Vue building block

3.1 Boîte noire

Chaque bloc est fermé. Les noms qui suivent sont les interfaces qu'il fournit.

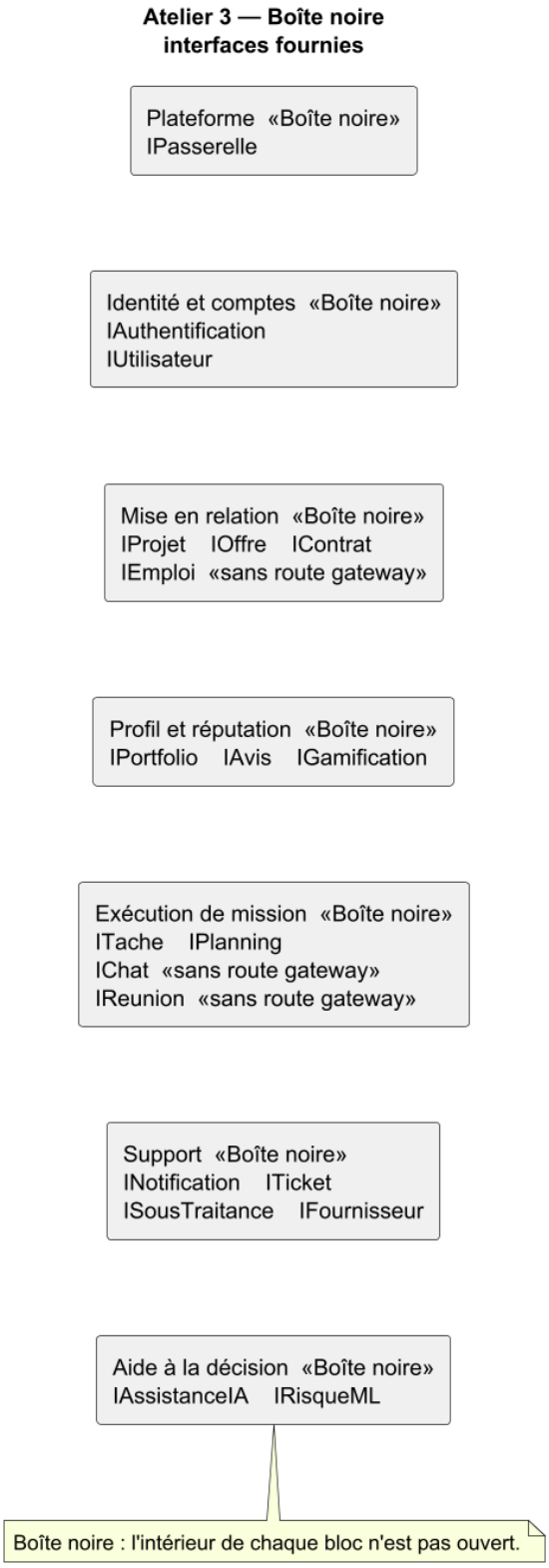


Figure 2. Paquets en boîte noire et interfaces fournies.

3.2 Dépendances

La passerelle retire le premier segment d'URL (*StripPrefix=1*). Les flèches de la figure 4 n'existent que si le code contient l'appel.

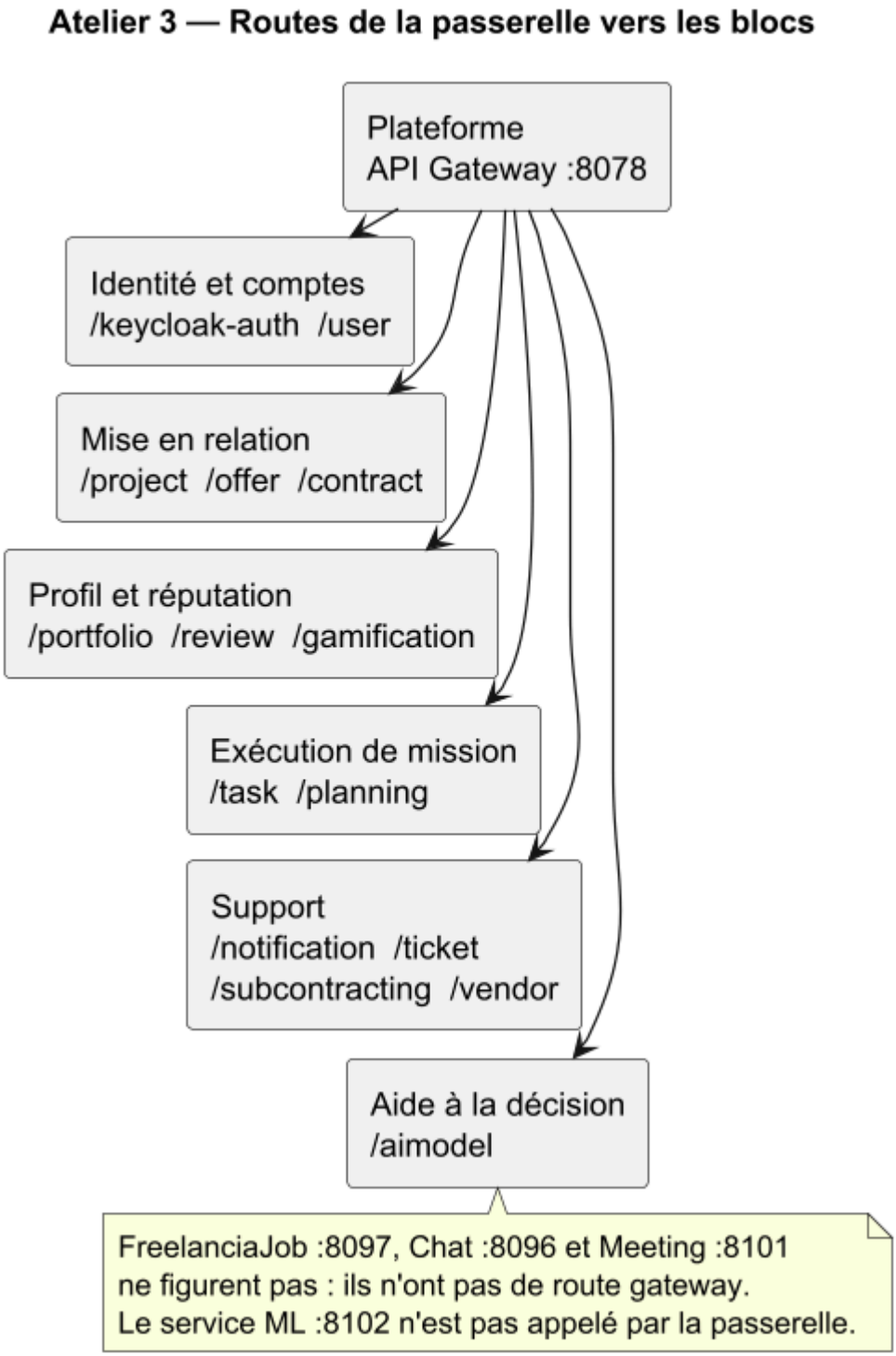


Figure 3. Routes de la passerelle vers les blocs.

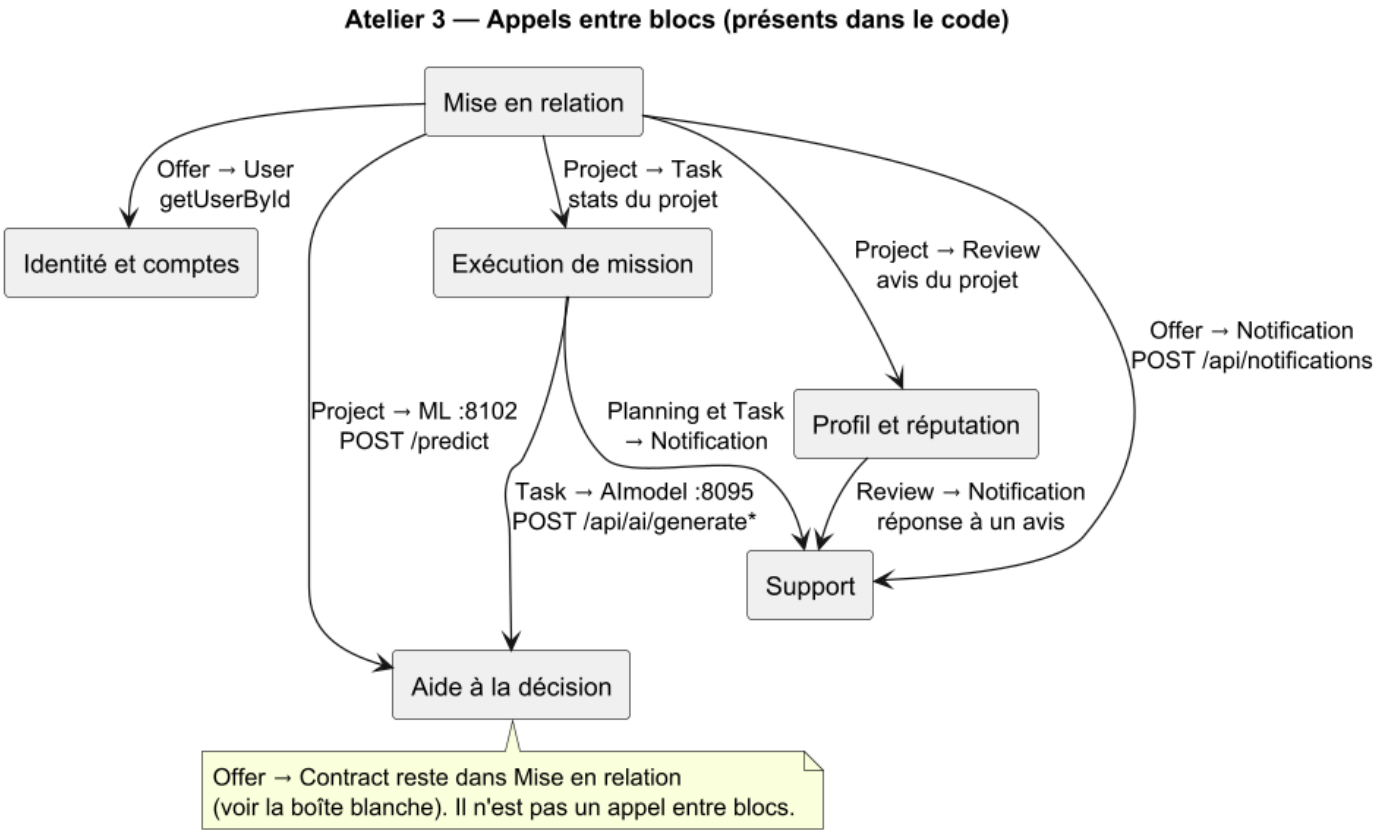


Figure 4. Appels entre blocs. Offer vers Contract reste interne à Mise en relation.

3.3 Boîte blanche de Mise en relation

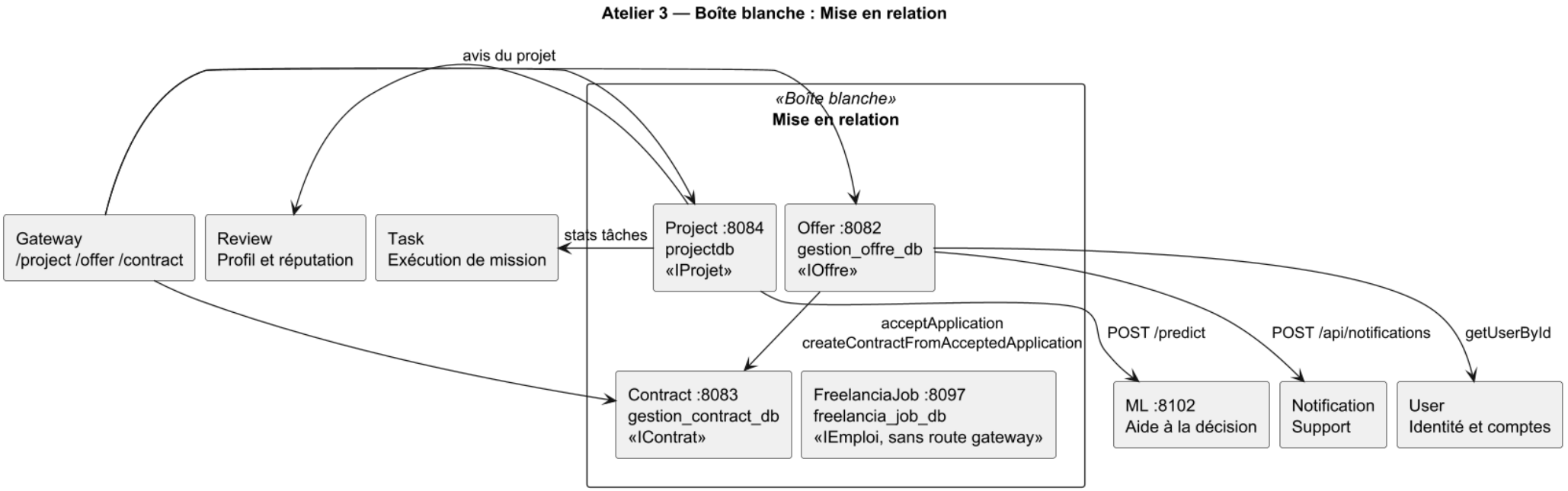


Figure 5. Sous-paquets, composants, interfaces et dépendances de Mise en relation. L'acceptation d'une candidature appelle ContractClient.createContractFromAcceptedApplication.

3.4 Classes et méthodes

Seuls Project et Offer sont ouverts jusqu'aux classes. Les méthodes sont celles des types du dépôt. Les autres blocs restent au niveau composant.

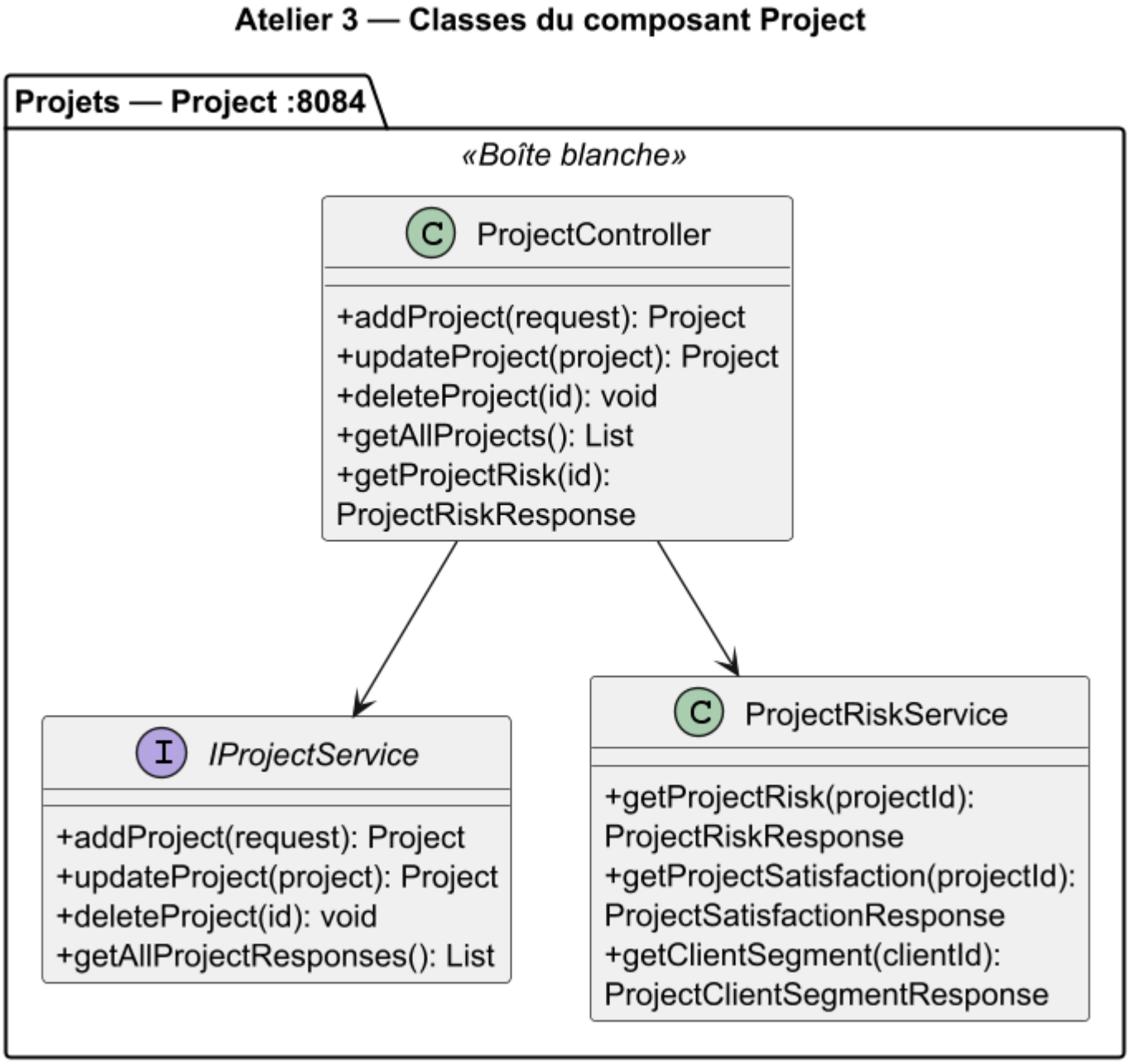


Figure 6. ProjectController, IProjectService et ProjectRiskService.

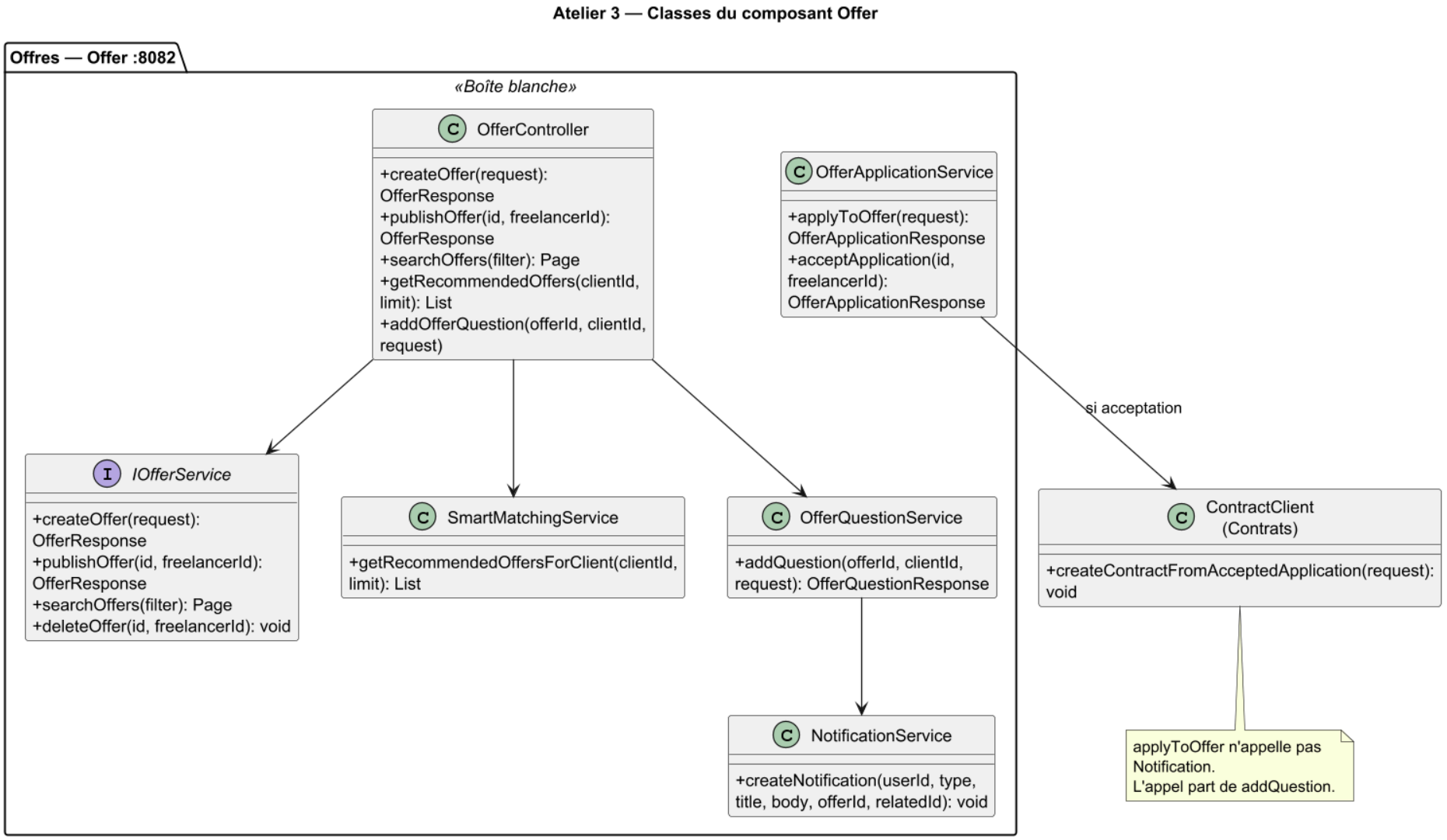


Figure 7. OfferController, IOfferService, candidature, question et NotificationService. Le lien vers ContractClient ne part que de l'acceptation.

4. Atelier 4 — Contexte, déploiement, runtime

4.1 Vue de contexte

Les acteurs n'entrent que par Plateforme. Keycloak, MySQL, Firebase, Ollama, GitHub, Google Calendar et Google Translate sont hors des sept blocs.

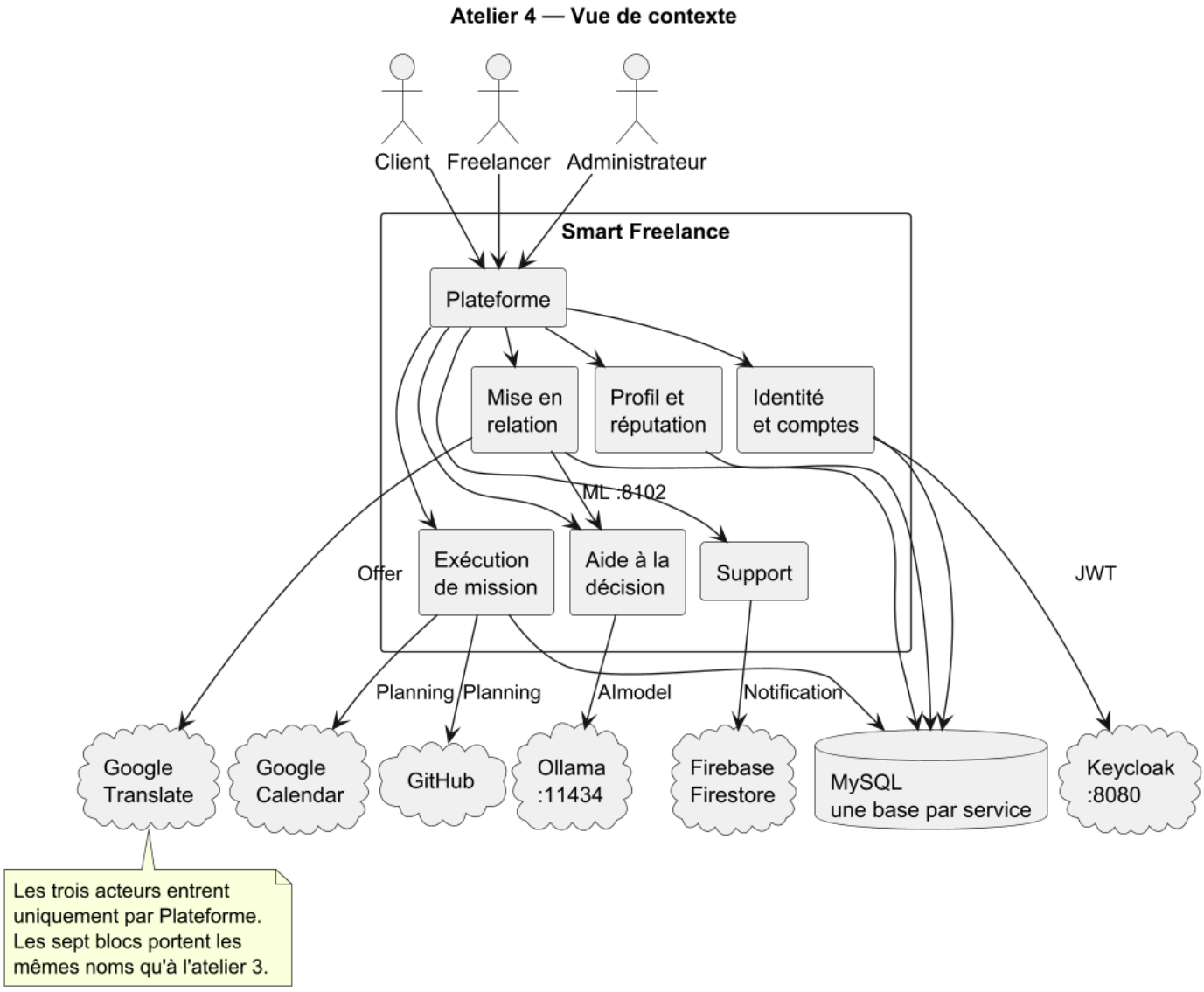


Figure 8. Vue de contexte. Les flèches Planning partent d'Exécution de mission vers GitHub et Google Calendar. La flèche Offer part de Mise en relation vers Google Translate. La flèche ML :8102 part de Mise en relation vers Aide à la décision.

4.2 Vue de déploiement

Le schéma décrit le dossier *k8s/*, namespace *smart-freelance*. Les nœuds portent les noms des blocs de l'atelier 3. Keycloak et Ollama sont dans le cluster, mais hors de ces blocs métier.

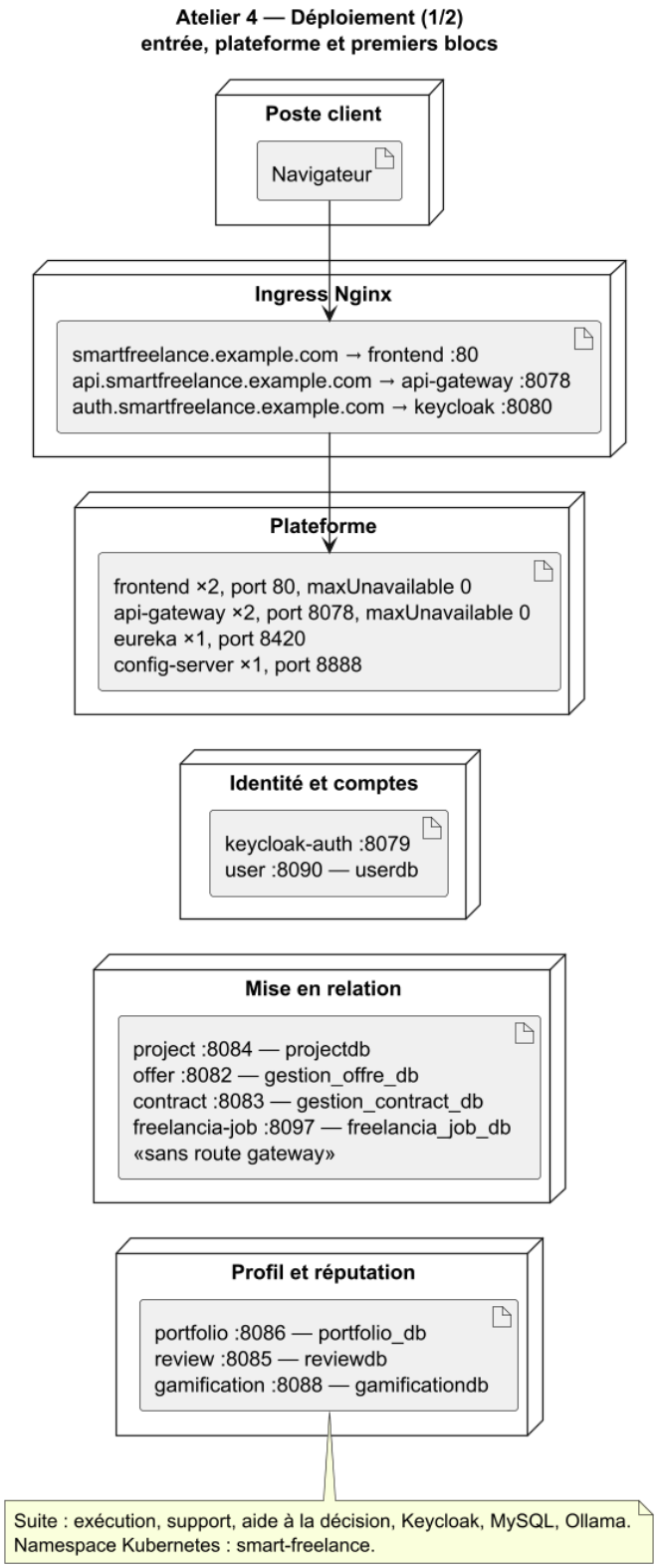


Figure 9. Déploiement, entrée et premiers blocs. Le navigateur atteint le frontend et la passerelle par l'ingress. L'hôte auth atteint Keycloak, montré à la figure suivante.

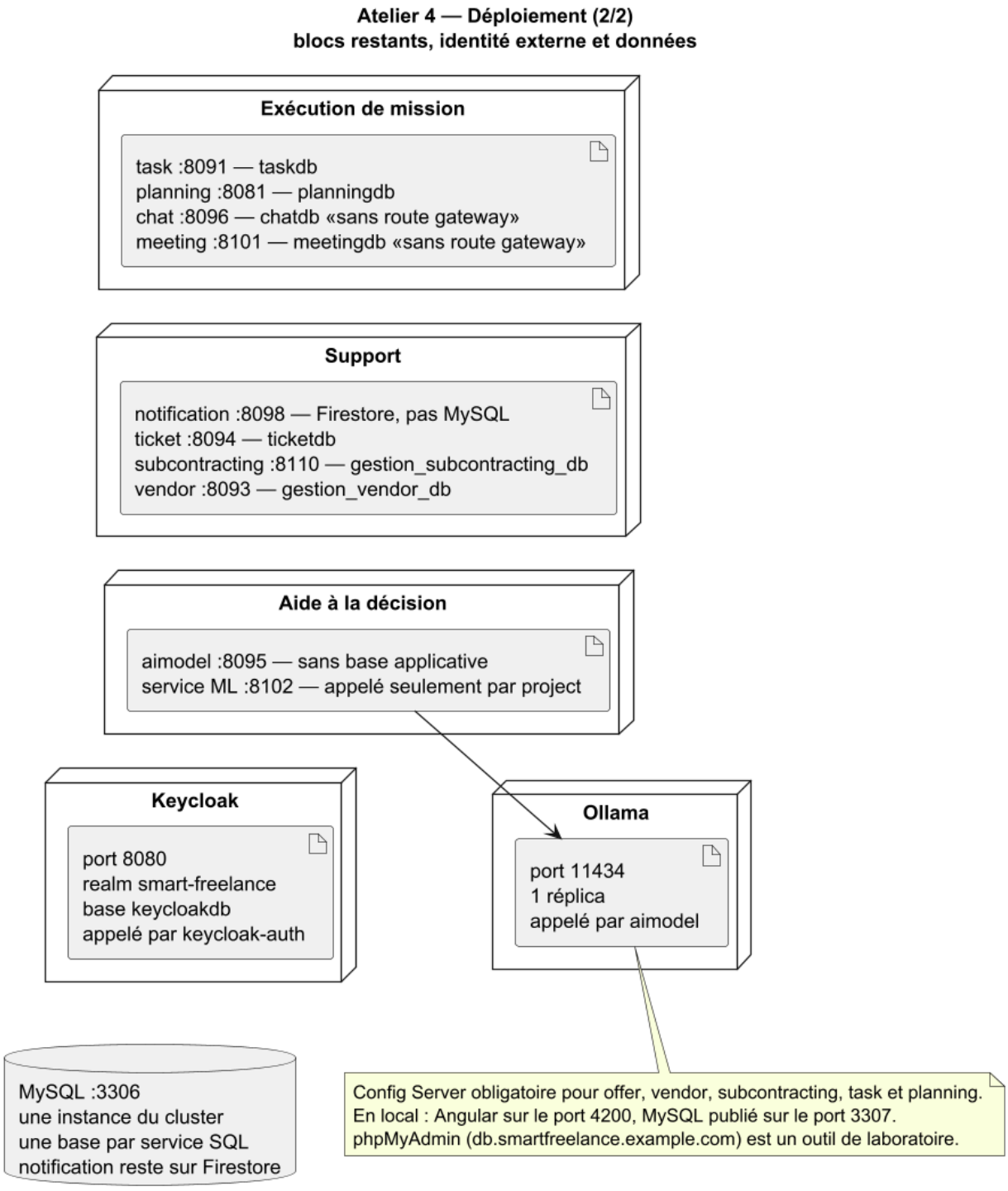


Figure 10. Déploiement, blocs restants, Keycloak, MySQL :3306 et Ollama :11434. En local, Angular écoute 4200 et MySQL est publié sur 3307.

Service	Port	Préfixe gateway	Donnée
Eureka	8420	—	—
Config Server	8888	—	obligatoire pour offer, vendor, subcontracting, task, planning
API Gateway	8078	—	—
keycloak-auth	8079	/keycloak-auth	Keycloak :8080
user	8090	/user	userdb
planning	8081	/planning	planningdb
offer	8082	/offer	gestion_offre_db
contract	8083	/contract	gestion_contract_db
project	8084	/project	projectdb
review	8085	/review	reviewdb
portfolio	8086	/portfolio	portfolio_db
gamification	8088	/gamification	gamificationdb
task	8091	/task	taskdb
vendor	8093	/vendor	gestion_vendor_db
ticket	8094	/ticket	ticketdb
aimodel	8095	/aimodel	Ollama, pas de base applicative
chat	8096	aucune	chatdb
freelancia-job	8097	aucune	freelancia_job_db
notification	8098	/notification	Firestore
meeting	8101	aucune	meetingdb
service ML	8102	aucune	appel serveur, depuis project
subcontracting	8110	/subcontracting	gestion_subcontracting_db

4.3 Vue runtime

Deux scénarios. Les lignes de vie utilisent les blocs de l'atelier 3.

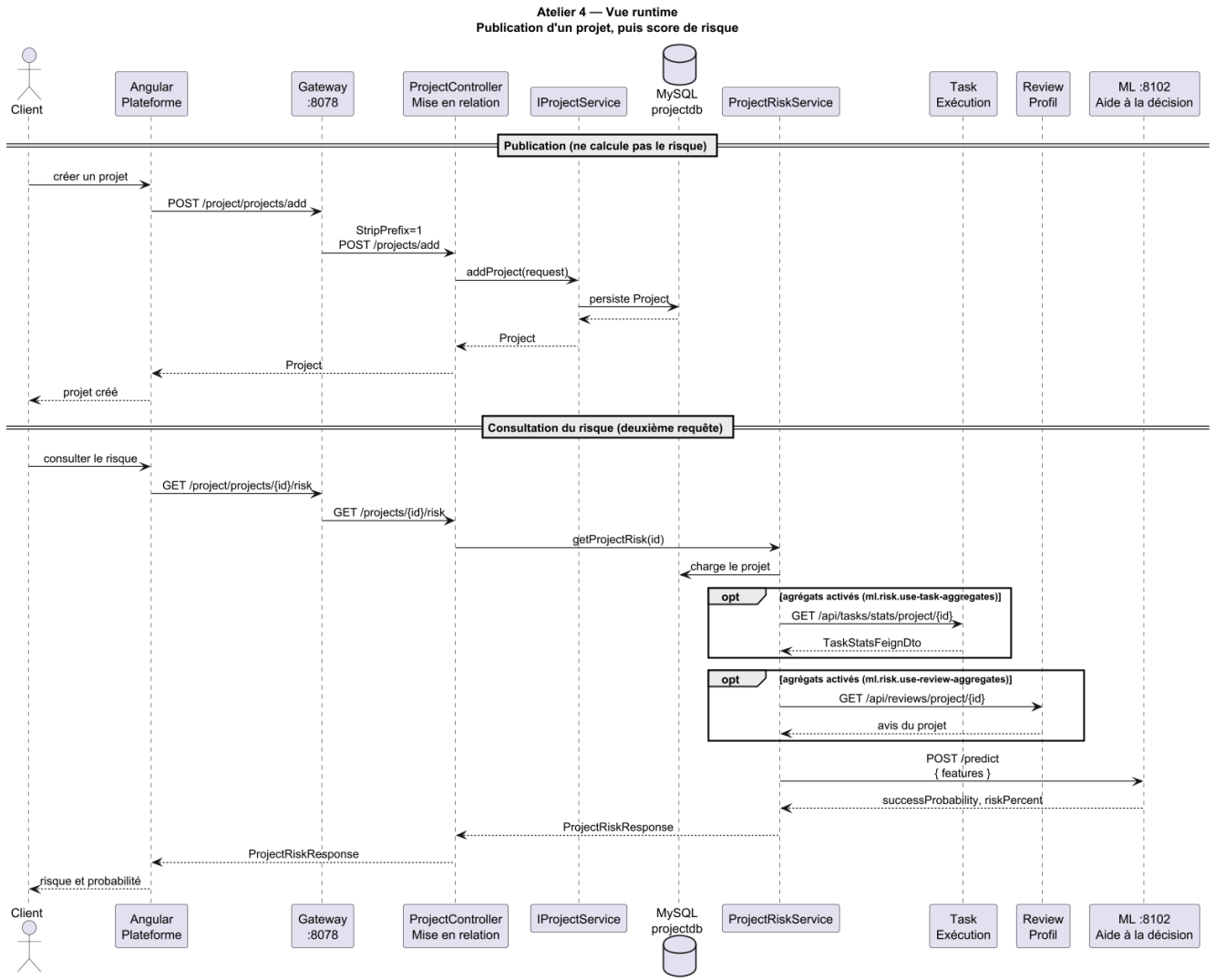


Figure 11. Le client publie un projet (POST /project/projects/add). Cet appel ne calcule pas le risque. Une deuxième requête, GET /project/projects/{id}/risk, charge le projet, interroge Task et Review si les agrégats sont activés, puis envoie POST /predict au service ML :8102.

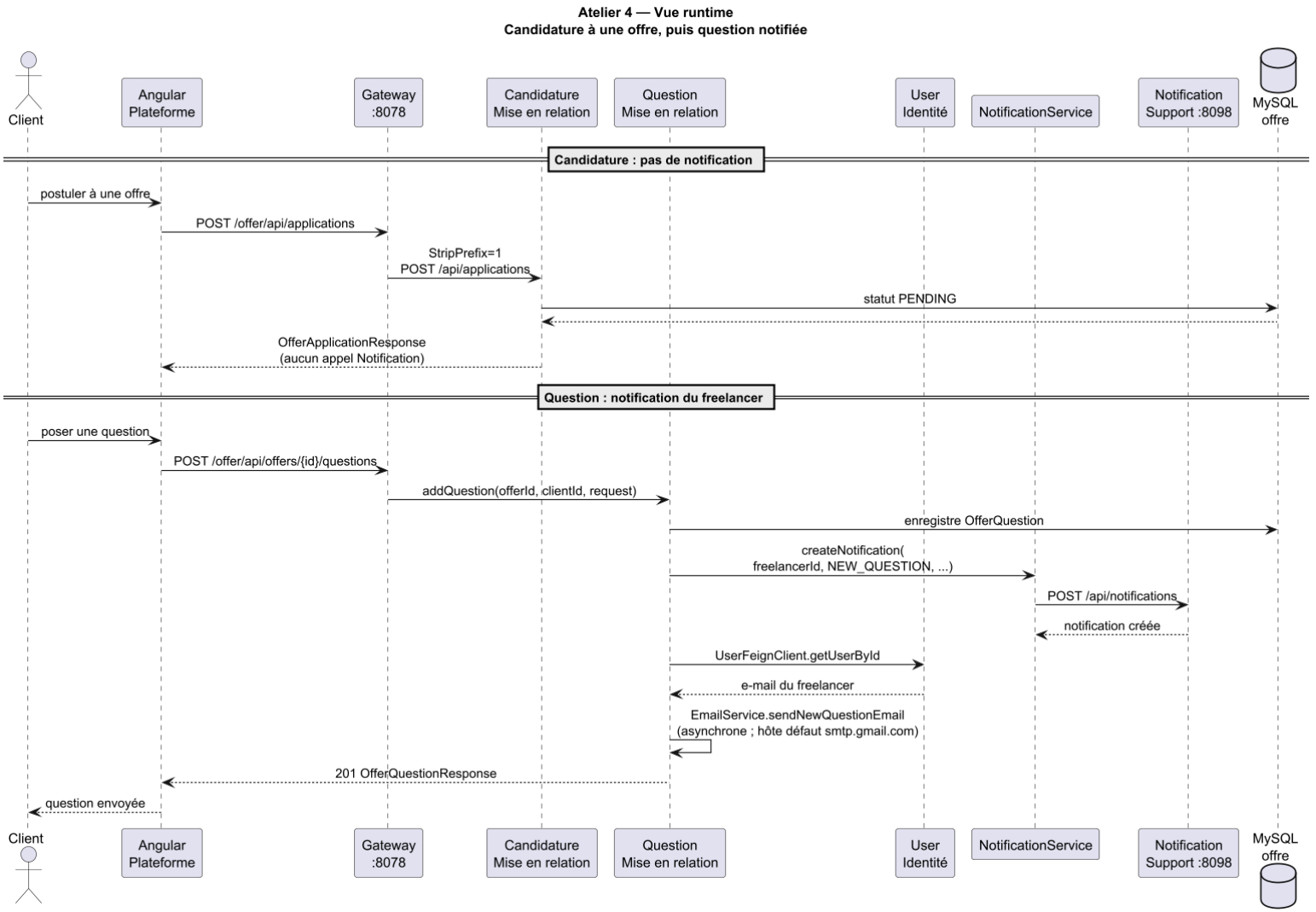


Figure 12. Le client postule (POST /offer/api/applications) : la candidature est PENDING et Notification n'est pas appelé. Ensuite il pose une question (POST /offer/api/offers/{id}/questions). OfferQuestionService enregistre la question, appelle NotificationService.createNotification (POST /api/notifications, port 8098), relit l'utilisateur, puis envoie un e-mail SMTP si le mail est activé. L'hôte par défaut du service Offer est smtp.gmail.com.

Les lifelines « Candidature » et « Question » sont OfferApplicationService et OfferQuestionService, dans le bloc Mise en relation. La base de la figure 12 est gestion_offre_db.